TASK 13
Deploying Deep Learning Applications on Kubernetes

CIFAR-10 image classification API using TensorFlow, Flask, Docker and Minikube

Submitted By: Satyam Yadav

Programme: M.Sc. Data Science & Big Data Analyst

Assignment: Task 13: Deploying Deep Learning Applications on Kubernetes

Model artifact: cifar10_model.keras

Container image: cifar10-api:v1

Deployment: cifar10-deployment

Service: cifar10-service (NodePort 30080 → port 5000)

Environment: Windows PowerShell, Docker Desktop, Minikube, kubect

GitHub Repository: https://github.com/228263satyam/task13-kubernetes.git

1. Executive Summary

This project deployed the trained CIFAR-10 model cifar10_model.keras as a Flask API inside Docker image cifar10-api:v1. The image was loaded into Minikube and deployed through Kubernetes Deployment cifar10-deployment and NodePort Service cifar10-service. Homepage and health checks succeeded. A POST request to /predict returned class airplane (class_id 0) with a score of approximately 0.4618. After startup/readiness/liveness probe adjustments, the final status screenshot showed one ready replica and a running pod with zero restarts.

2. Objective

Create Kubernetes deployment manifests, configure a service, deploy the containerized model, expose it externally in the local cluster, and verify API accessibility and inference.

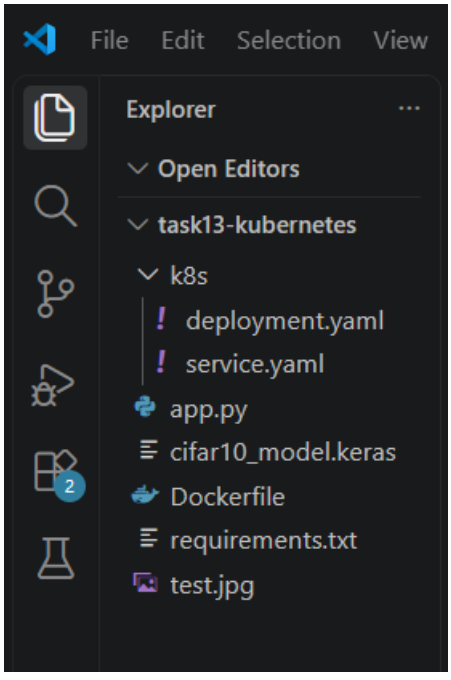
3. Tools and Technologies

Technology	Purpose
Python / TensorFlow / Keras	Load the trained model and run inference.
Flask	Provide homepage, health and prediction HTTP routes.
Gunicorn	Serve the Flask application in the container.
Docker	Package application code, dependencies and model.
Minikube	Run the local Kubernetes cluster.
kubectl	Apply manifests and inspect resources.
PowerShell, browser, curl.exe	Run commands and test the API.

4. System Architecture

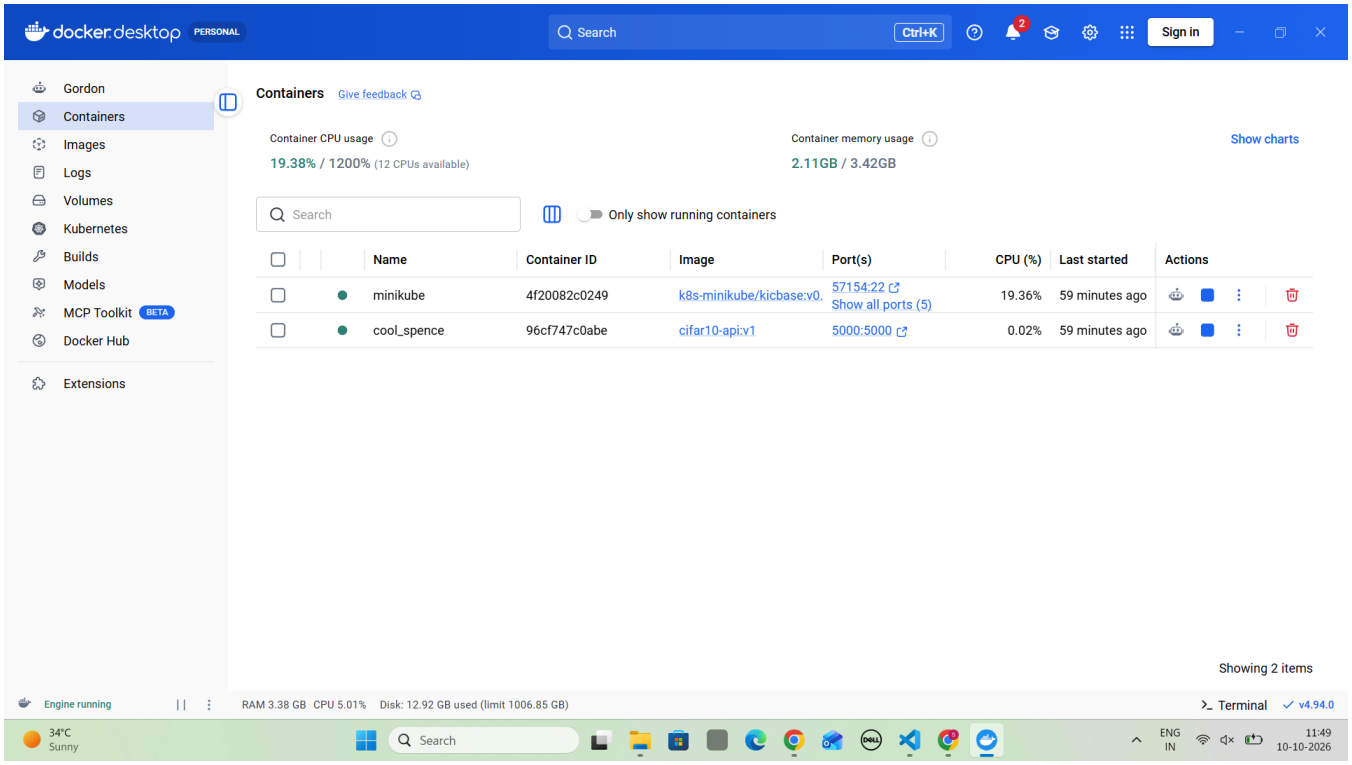
Client → Minikube NodePort Service → Kubernetes Pod → Gunicorn/Flask → TensorFlow/Keras model → JSON response. Service NodePort 30080 maps to container port 5000. Keep the Minikube service tunnel terminal open while testing its printed localhost URL.

5. Project Structure



6. Implementation Procedure

```
docker build -t cifar10-api:v1 .\nminikube image load cifar10-api:v1\nminikube image ls\nkubectl apply -f
k8s/deployment.yaml\nkubectl apply -f k8s/service.yaml\nkubectl rollout status deployment/cifar10-deployment --
timeout=300s\nkubectl get nodes\nkubectl get deployments\nkubectl get pods\nkubectl get services\nminikube service
cifar10-service
```



7. Application and Configuration Code

The following code documents the API and manifests used. Ensure image resizing and normalization match the preprocessing used to train the model.

7.1 app.py

```
from flask import Flask, request, jsonify
from tensorflow.keras.models import load_model
from PIL import Image
import numpy as np

app = Flask(__name__)
model = load_model("cifar10_model.keras")
classes = ["airplane", "automobile", "bird", "cat", "deer",
           "dog", "frog", "horse", "ship", "truck"]

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "CIFAR-10 Deep Learning API is running"})

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "healthy"})

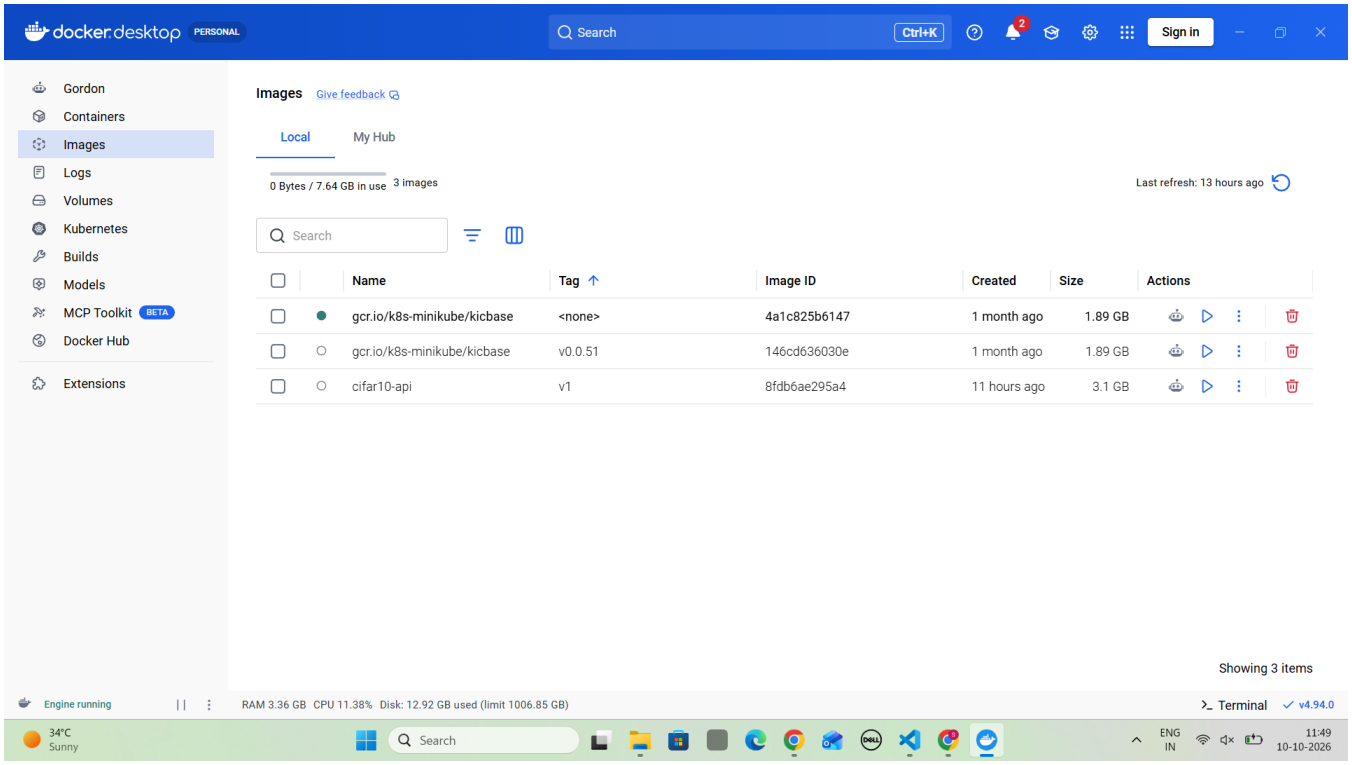
@app.route("/predict", methods=["POST"])
def predict():
    if "image" not in request.files:
        return jsonify({"error": "Upload an image using the image field"}), 400
    image = Image.open(request.files["image"].stream).convert("RGB")
    height, width = model.input_shape[1], model.input_shape[2]
    image = image.resize((width, height))
    array = np.array(image, dtype=np.float32) / 255.0
    array = np.expand_dims(array, axis=0)
```

```
    predictions = model.predict(array, verbose=0)
    class_id = int(np.argmax(predictions[0]))
    return jsonify({"class_id": class_id, "prediction": classes[class_id],
                    "confidence": float(predictions[0][class_id])})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

7.2 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY cifar10_model.keras .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```



7.3 requirements.txt

```
tensorflow\nflask\npillow\nnumpy\ngunicorn
```

7.4 k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cifar10-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cifar10-api
  template:
    metadata:
      labels:
        app: cifar10-api
    spec:
```

```
    containers:
      - name: cifar10-api
        image: cifar10-api:v1
        imagePullPolicy: IfNotPresent
        ports:
          - containerPort: 5000
        startupProbe:
          httpGet:
            path: /health
            port: 5000
          periodSeconds: 10
          failureThreshold: 30
        readinessProbe:
          httpGet:
            path: /health
            port: 5000
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 3
        livenessProbe:
          httpGet:
            path: /health
            port: 5000
          periodSeconds: 20
          timeoutSeconds: 5
          failureThreshold: 3
```

7.5 k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: cifar10-service
spec:
  type: NodePort
  selector:
    app: cifar10-api
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30080
```

8. Testing and Results

Test	Observed result	Status
Homepage GET /	API running message returned	Pass
Health GET /health	JSON status healthy	Pass
Image in Minikube	cifar10-api:v1 listed	Pass
Deployment rollout	Rollout successfully completed	Pass
Pod status	Final evidence: Running, Ready 1/1, zero restarts	Pass
Service	NodePort 30080 → target port 5000	Pass
Prediction POST /predict	class_id 0; airplane; score ≈ 0.461775	Pass

The confidence value is the model's output score and should not automatically be treated as a calibrated probability.

9. Troubleshooting

The first pod experienced health-probe timeouts and restarted; the previous instance showed exit code 137. A startupProbe was added and the readiness/liveness timeout was increased from one to five seconds. The rollout then succeeded, and the later pod status showed Running, Ready 1/1 and zero restarts. CUDA-not-found messages indicate CPU inference and are not necessarily a failure.

10. Screenshot Evidence

Screenshots provided during implementation are included below. The earlier not-ready state is retained as troubleshooting evidence; the later screenshot documents the corrected state.

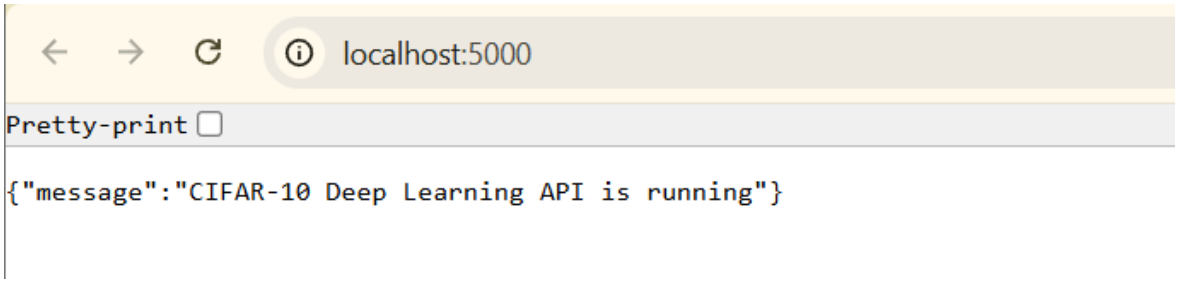


Figure 1. API homepage: the CIFAR-10 API is running.

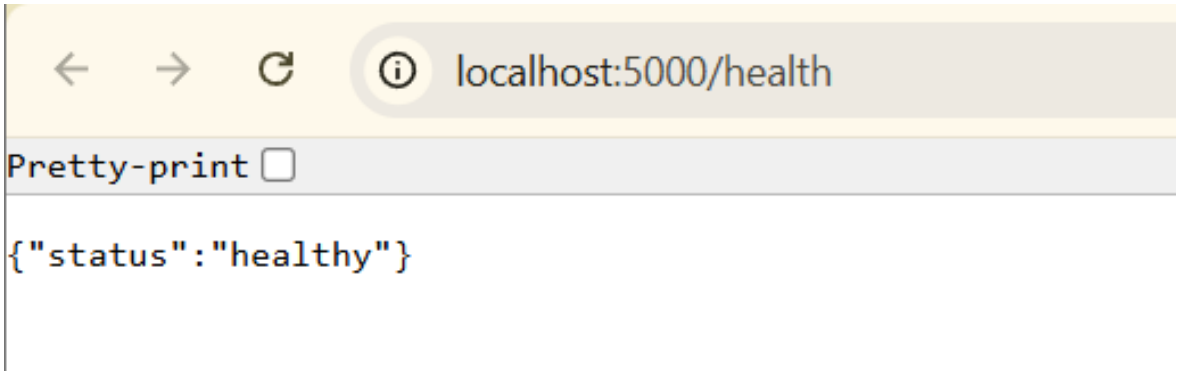


Figure 2. Health endpoint returns status healthy.

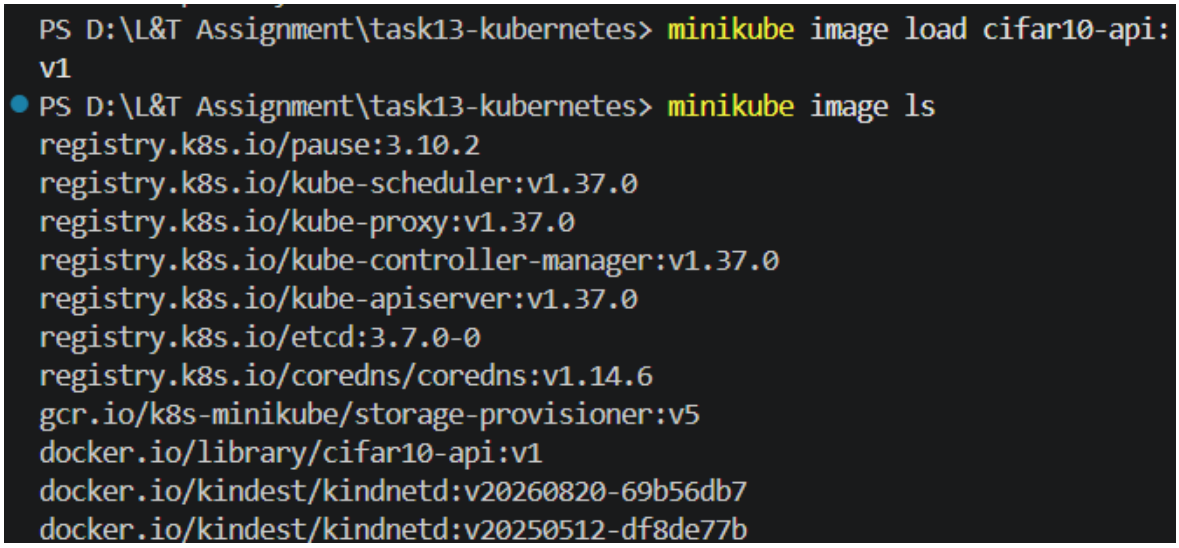


Figure 3. Minikube image listing includes cifar10-api:v1.

```
PS D:\L&T Assignment\task13-kubernetes> kubectl apply -f k8s/deployment.
yaml
deployment.apps/cifar10-deployment created
PS D:\L&T Assignment\task13-kubernetes> kubectl apply -f k8s/service.yam
l
service/cifar10-service created
PS D:\L&T Assignment\task13-kubernetes>
```

Figure 4. Kubernetes Deployment and Service resources created.

```
PS D:\L&T Assignment\task13-kubernetes> kubectl get deployments
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
cifar10-deployment   1/1     1            1           74s
PS D:\L&T Assignment\task13-kubernetes> kubectl get pods
NAME                                  READY   STATUS    RESTARTS   AGE
cifar10-deployment-7d85b57fcb-h5bdf   1/1     Running   0          80s
PS D:\L&T Assignment\task13-kubernetes> kubectl get services
NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
     AGE
cifar10-service   NodePort    10.103.21.181   <none>        5000:30080/T
CP   65s
kubernetes        ClusterIP   10.96.0.1       <none>        443/TCP
     10h
PS D:\L&T Assignment\task13-kubernetes>
```

Figure 5. Initial not-ready state captured during troubleshooting.

```
PS D:\L&T Assignment\task13-kubernetes> minikube service cifar10-service
```

NAMESPACE	NAME	TARGET PORT	URL
default	cifar10-service	5000	http://192.168.49.2:30080

```
Starting tunnel for service cifar10-service.
```

NAMESPACE	NAME	TARGET PORT	URL
default	cifar10-service		http://127.0.0.1:50952

```
Opening service default/cifar10-service in default browser...
Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Figure 6. Minikube NodePort service and local tunnel URL.

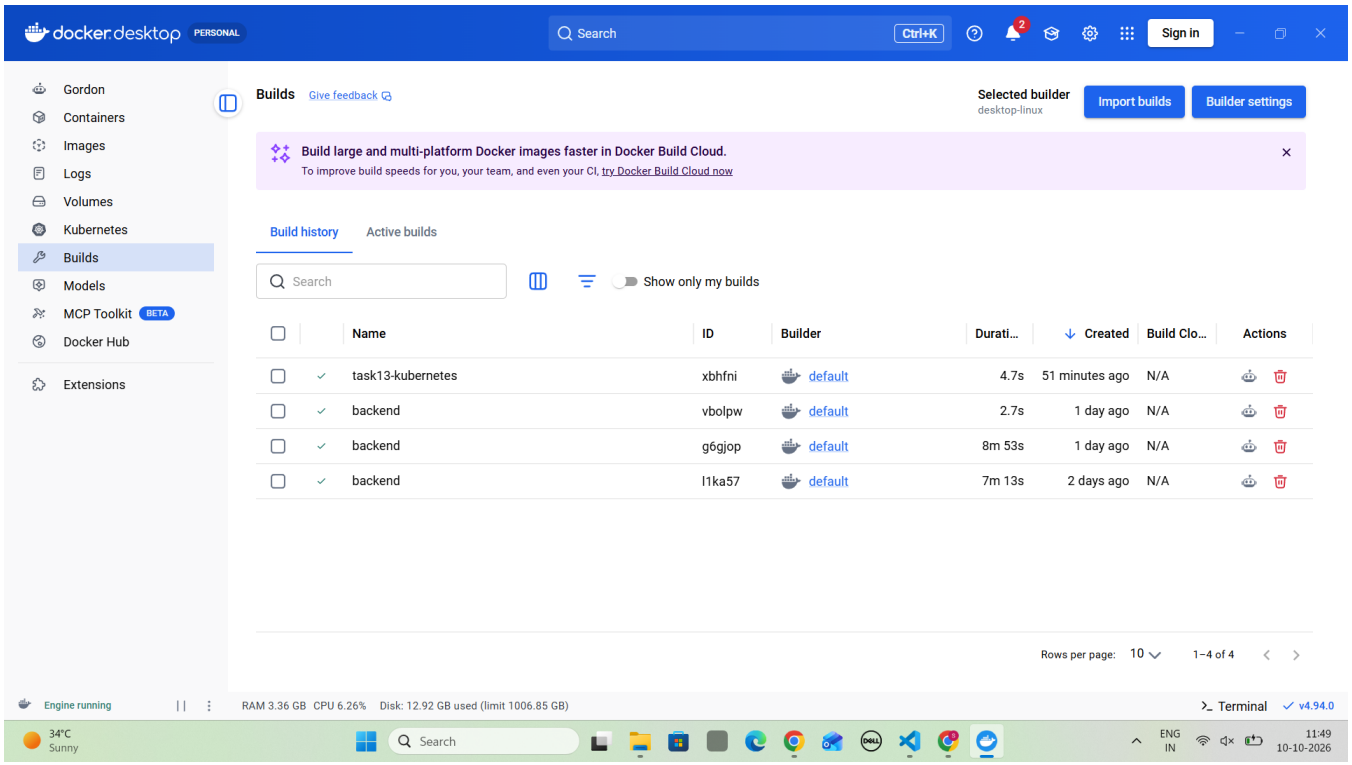
```
PS D:\L&T Assignment\task13-kubernetes> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
cifar10-deployment  0/1     1            0           7m50s
PS D:\L&T Assignment\task13-kubernetes> kubectl get pods
NAME                                 READY   STATUS    RESTARTS      AGE
cifar10-deployment-7d85b57fcb-h5bdf  0/1     Running   1 (10s ago)   8m5s
PS D:\L&T Assignment\task13-kubernetes> kubectl get services
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
cifar10-service  NodePort    10.103.21.181   <none>        5000:30080/TCP   7m47s
kubernetes       ClusterIP   10.96.0.1       <none>        443/TCP          10h
PS D:\L&T Assignment\task13-kubernetes>
```

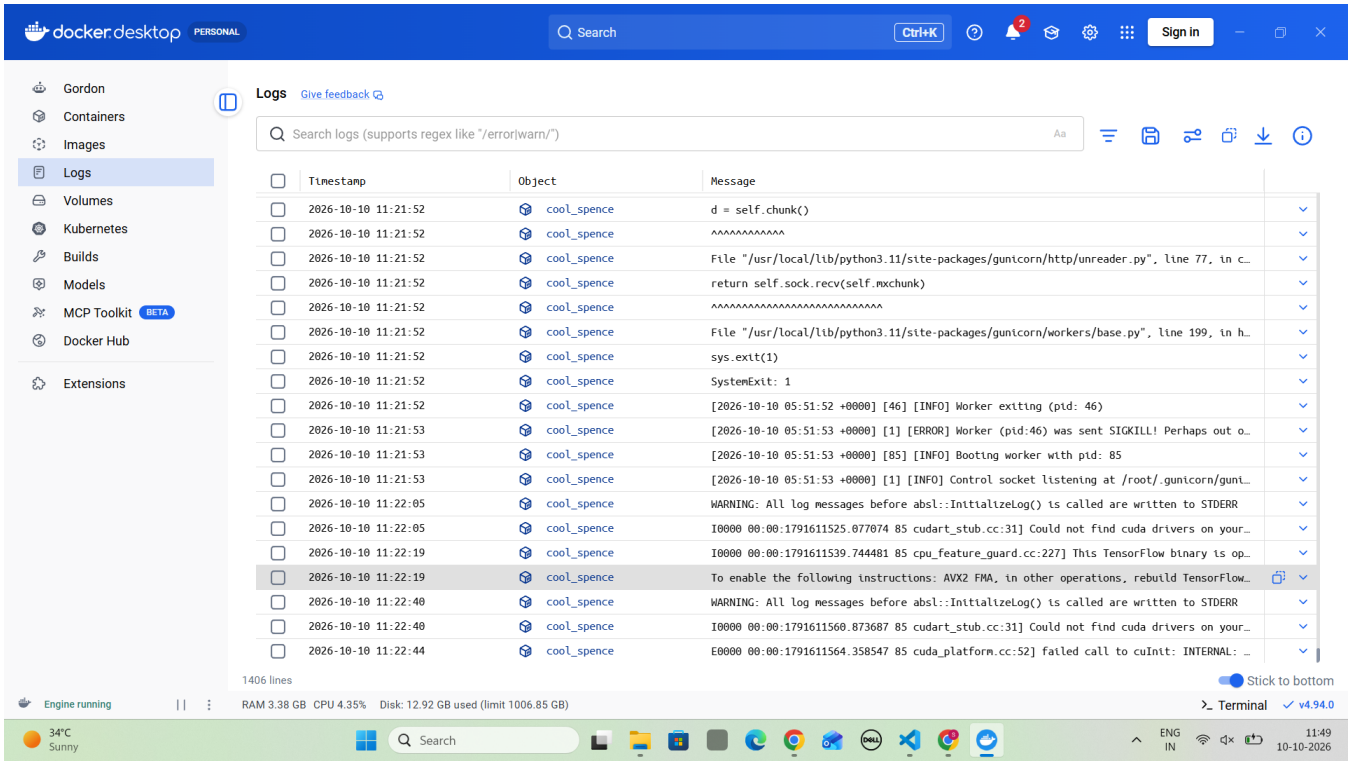
Figure 7. Corrected rollout: one ready replica and pod running with zero restarts.

```
PS D:\L&T Assignment\task13-kubernetes> kubectl get deployments

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
cifar10-deployment  1/1     1            1           16m
PS D:\L&T Assignment\task13-kubernetes> kubectl get pods
NAME                                 READY   STATUS    RESTARTS   AGE
cifar10-deployment-6cb98b96b7-nn28q  1/1     Running   0          2m2s
PS D:\L&T Assignment\task13-kubernetes>
```

Figure 8. Additional final verification screenshot.





11. Conclusion

The CIFAR-10 model was packaged into a Docker image, loaded into Minikube, deployed with Kubernetes and exposed through a NodePort Service. Homepage and health requests succeeded, and a prediction request returned airplane. The final status showed one ready replica and a running pod. The task demonstrated containerization, Kubernetes manifests, service exposure, health checks and inference testing.